

LAB 01 REPORT

Kernel Configuration & Booting an Embedded Linux System (ARMv7)

Course: Embedded Linux Systems

Section 1. Student Information

Full Name	Trần Gia Phúc
Student ID (MSSV)	SE204417
Group	[Fill in group, if any]
Date Conducted	12/09/2026
Class / Course	Embedded Linux Systems

Section 2. Boot Sequence Diagram (required)

Diagram showing the complete boot sequence of the system, from power-on until reaching the BusyBox shell:

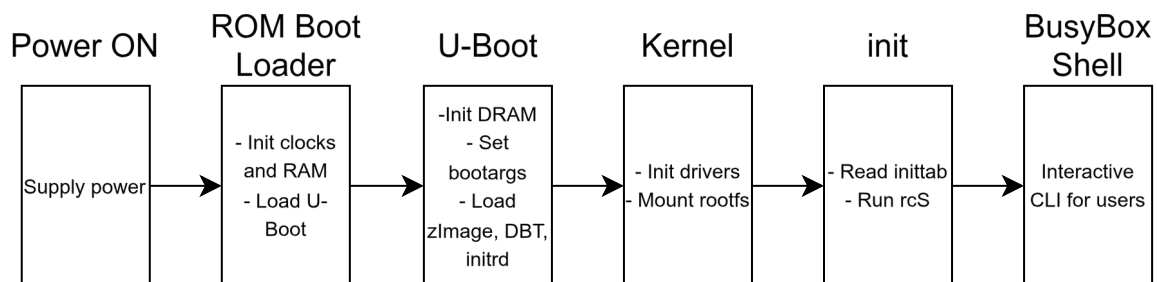


Table explaining each step in detail (fill in all 5 rows corresponding to the 5 stages):

Stage	Component Running	Main Task	What Is Passed to the Next Step
Power ON	QEMU vexpress-a9 and the ARM Cortex-A9 CPU are initialized	Initializes the hardware model and CPU, starts the system boot process	Execution control is handed over to the bootloader
ROM bootloader	Boot/ROM mechanism of the emulated system	Determines the boot source and hands control to the bootloader for execution	Hands over execution to U-Boot
U-Boot	U-Boot	Initializes the necessary components, sets up bootargs, loads the Linux Kernel into RAM and executes the boot command	Kernel image + bootargs + rootfs information
Kernel	Linux Kernel ARMv7 (zImage)	Initializes the CPU, memory, drivers, and kernel components; finds and mounts the root filesystem	Hands over execution to /init
init → BusyBox shell	/init and BusyBox	init initializes user	Displays the BusyBox

		space and runs the BusyBox shell	shell for the user to run commands
--	--	----------------------------------	------------------------------------

Section 3. Practical Logs

Part A — Build Linux Kernel

- Output of the make vexpress_defconfig command

```
phuc@phuc-Inspiron-14-5430:~/embedded_lab1/kernel/linux-5.15$ cd ~/embedded_lab1/kernel/li
nux-5.15

# 1. Dọn sạch hoàn toàn
make mrproper

# 2. Cấu hình lại
export ARCH=arm
export CROSS_COMPILE=arm-linux-gnueabi-
make vexpress_defconfig

# 3. Compile với ép đúng kiến trúc ARMv7-A
make -j$(nproc) KCFLAGS="-march=armv7-a" zImage dtbs
CLEAN    drivers/scsi
CLEAN    drivers/tty/vt
CLEAN    drivers/video/logo
CLEAN    kernel
CLEAN    lib
CLEAN    usr
CLEAN    vmlinux.symvers modules.builtin modules.builtin.modinfo
CLEAN    scripts/basic
CLEAN    scripts/dtc
CLEAN    scripts/kconfig
CLEAN    scripts/mod
CLEAN    scripts
CLEAN    include/config include/generated .config .config.old .version
HOSTCC   scripts/basic/fixdep
HOSTCC   scripts/kconfig/conf.o
HOSTCC   scripts/kconfig/confdata.o
HOSTCC   scripts/kconfig/expr.o
LEX       scripts/kconfig/lexer.lex.c
YACC     scripts/kconfig/parser.tab.[ch]
HOSTCC   scripts/kconfig/lexer.lex.o
HOSTCC   scripts/kconfig/menu.o
HOSTCC   scripts/kconfig/parser.tab.o
HOSTCC   scripts/kconfig/preprocess.o
HOSTCC   scripts/kconfig/symbol.o
HOSTCC   scripts/kconfig/util.o
HOSTLD   scripts/kconfig/conf
#
# configuration written to .config
#
```

Figure 3.1: make vexpress_defconfig output.

- Final log of the kernel build process (make zImage dtbs modules)

```

SORTTAB vmlinux
SYSMAP System.map
OBJCOPY arch/arm/boot/Image
Kernel: arch/arm/boot/Image is ready
LDS arch/arm/boot/compressed/vmlinux.lds
AS arch/arm/boot/compressed/head.o
GZIP arch/arm/boot/compressed/piggy_data
CC arch/arm/boot/compressed/misc.o
CC arch/arm/boot/compressed/decompress.o
CC arch/arm/boot/compressed/string.o
AS arch/arm/boot/compressed/hyp-stub.o
CC arch/arm/boot/compressed/fdt_rw.o
CC arch/arm/boot/compressed/fdt_ro.o
CC arch/arm/boot/compressed/fdt_wip.o
CC arch/arm/boot/compressed/fdt.o
CC arch/arm/boot/compressed/fdt_check_mem_start.o
AS arch/arm/boot/compressed/lib1funcs.o
AS arch/arm/boot/compressed/ashldi3.o
AS arch/arm/boot/compressed/bswapsdi2.o
AS arch/arm/boot/compressed/piggy.o
LD arch/arm/boot/compressed/vmlinux
/usr/bin/arm-linux-gnueabi-ld.bfd: warning: arch/arm/boot/compressed/vmlinux has a LOAD
segment with RWX permissions
OBJCOPY arch/arm/boot/zImage
Kernel: arch/arm/boot/zImage is ready
phuc@phuc-Inspiron-14-5430:~/embedded_lab1/kernel/linux-5.15$

```

Figure 3.2: make zImage dtbs modules result.

- Result of the `ls -lh arch/arm/boot/zImage` command

```

phuc@phuc-Inspiron-14-5430:~/embedded_lab1/kernel/linux-5.15$ ls -lh arch/arm/boot/zImage
ls -lh arch/arm/boot/dts/vexpress-v2p-ca9.dtb
-rwxrwxr-x 1 phuc phuc 4.9M Sep 12 12:21 arch/arm/boot/zImage
-rw-rw-r-- 1 phuc phuc 14K Sep 12 12:18 arch/arm/boot/dts/vexpress-v2p-ca9.dtb
phuc@phuc-Inspiron-14-5430:~/embedded_lab1/kernel/linux-5.15$

```

Figure 3.3: ls -lh arch/arm/boot/zImage result.

Part B — Configuring and Using U-Boot

- Output of the `printenv` command

```

=> printenv
arch=arm
baudrate=38400
board=vexpress
board_name=vexpress
boot_a_script=load ${devtype} ${devnum}:${distro_bootpart} ${scriptaddr} ${prefix}${script}
}; source ${scriptaddr}
boot_efi_binary=load ${devtype} ${devnum}:${distro_bootpart} ${kernel_addr_r} efi/boot/boottarm.efi; if fdt addr ${fdt_addr_r}; then bootefi ${kernel_addr_r} ${fdt_addr_r};else bootefi ${kernel_addr_r} ${fdtcontroladdr};fi
boot_efi_bootmgr=if fdt addr ${fdt_addr_r}; then bootefi bootmgr ${fdt_addr_r};else bootefi bootmgr;fi
boot_extlinux=sysboot ${devtype} ${devnum}:${distro_bootpart} any ${scriptaddr} ${prefix}${boot_syslinux_conf}
boot_prefixes=/ /boot/
boot_script_dhcp=boot.scr.uimg
boot_scripts=boot.scr.uimg boot.scr
boot_syslinux_conf=extlinux/extlinux.conf
boot_targets=mmc1 mmc0 pxe dhcp
bootargs=console=tty0 console=ttyAMA0,38400n8
bootcmd=run distro_bootcmd; run bootflash
bootcmd_dhcp=devtype=dhcp; if dhcp ${scriptaddr} ${boot_script_dhcp}; then source ${scriptaddr}; fi;setenv efi_fdtfile ${fdtfile}; if test -z "${fdtfile}" -a -n "${soc}"; then setenv efi_fdtfile ${soc}-${board}${boardver}.dtb; fi; setenv efi_old_vci ${bootp_vci};setenv efi_old_arch ${bootp_arch};setenv bootp_vci PXEClient:Arch:00010:UNDI:003000;setenv bootp_arch 0xa;if dhcp ${kernel_addr_r}; then tftpboot ${fdt_addr_r} dtb/${efi_fdtfile};if fdt addr ${fdt_addr_r}; then bootefi ${kernel_addr_r} ${fdt_addr_r}; else bootefi ${kernel_addr_r} ${fdtcontroladdr};fi;fi;setenv bootp_vci ${efi_old_vci};setenv bootp_arch ${efi_old_arch};setenv efi_fdtfile;setenv efi_old_arch;setenv efi_old_vci;
bootcmd_mmc0=devnum=0; run mmc_boot
bootcmd_mmc1=devnum=1; run mmc_boot
bootcmd_pxe=dhcp; if pxe get; then pxe boot; fi
bootdelay=2
bootflash=run flashargs; cp ${ramdisk_addr} ${ramdisk_addr_r} ${maxramdisk}; bootm ${kernel_addr_r} ${ramdisk_addr_r}
console=ttyAMA0,38400n8
cpu=armv7
distro_bootcmd=for target in ${boot_targets}; do run bootcmd_${target}; done
dram=1024M
efi_dtb_prefixes=/ /dtb/ /dtb/current/
ethaddr=52:54:00:12:34:56
fdt_addr_r=0x60000000
fdtcontroladdr=7ffe0430
fdtfile=vexpress-v2p-ca9.dtb
flashargs=setenv bootargs root=${root} console=${console} mem=${dram} mtdparts=${mtd} mmc0 .fmax=190000 devtmpfs.mount=0 vmalloc=256M

```

Figure 3.4: printenv output (part 1).

```

flashargs=setenv bootargs root=${root} console=${console} mem=${dram} mtdparts=${mtd} mmci
.fmax=190000 devtmpfs.mount=0 vmlloc=256M
kernel_addr_r=0x60100000
load_efi_dtb=load ${devtype} ${devnum}:${distro_bootpart} ${fdt_addr_r} ${prefix}${efi_fdt
file}
loadaddr=0x90000000
mmc_boot=if mmc dev ${devnum}; then devtype=mmc; run scan_dev_for_boot_part; fi
mtd=armflash:1M@0x800000(uboot),7M@0x1000000(kernel),24M@0x2000000(initrd)
root=/dev/sda1 rw
scan_dev_for_boot=echo Scanning ${devtype} ${devnum}:${distro_bootpart}...; for prefix in
${boot_prefixes}; do run scan_dev_for_extlinux; run scan_dev_for_scripts; done;run scan_de
v_for_efi;
scan_dev_for_boot_part=part list ${devtype} ${devnum} -bootable devplist; env exists devpl
ist || setenv devplist 1; for distro_bootpart in ${devplist}; do if fstype ${devtype} ${de
vnum}:${distro_bootpart} bootfstype; then run scan_dev_for_boot; fi; done; setenv devplist
scan_dev_for_efi=setenv efi_fdtfile ${fdtfile}; if test -z "${fdtfile}" -a -n "${soc}"; th
en setenv efi_fdtfile ${soc}-${board}${boardver}.dtb; fi; for prefix in ${efi_dtb_prefixes
}; do if test -e ${devtype} ${devnum}:${distro_bootpart} ${prefix}${efi_fdtfile}; then run
load_efi_dtb; fi;done;run boot_efi_bootmgr;if test -e ${devtype} ${devnum}:${distro_bootp
art} efi/boot/bootarm.efi; then echo Found EFI removable media binary efi/boot/bootarm.efi
; run boot_efi_binary; echo EFI LOAD FAILED: continuing...; fi; setenv efi_fdtfile
scan_dev_for_extlinux=if test -e ${devtype} ${devnum}:${distro_bootpart} ${prefix}${boot_s
yslinux_conf}; then echo Found ${prefix}${boot_syslinux_conf}; run boot_extlinux; echo SCR
IPT FAILED: continuing...; fi
scan_dev_for_scripts=for script in ${boot_scripts}; do if test -e ${devtype} ${devnum}:${d
istro_bootpart} ${prefix}${script}; then echo Found U-Boot script ${prefix}${script}; run
boot_a_script; echo SCRIPT FAILED: continuing...; fi; done
stderr=serial
stdin=serial
stdout=serial
ubifs_boot=env exists bootubipart || env set bootubipart UBI; env exists bootubivol || env
set bootubivol boot; if ubi part ${bootubipart} && ubismount ubi${devnum}:${bootubivol};
then devtype=ubi; run scan_dev_for_boot; fi
vendor=armltd

Environment size: 4238/262140 bytes
=>

```

Figure 3.4: printenv output (part 2).

- Output of setenv bootargs + printenv bootargs confirmation

```

=> setenv bootargs 'console=ttyAMA0,115200 root=/dev/ram0 rw init=/sbin/init mem=512M'
=> printenv bootargs
bootargs=console=ttyAMA0,115200 root=/dev/ram0 rw init=/sbin/init mem=512M
=> setenv bootcmd 'bootz 0x60100000 0x64000000 0x60000000'
=> printenv bootcmd
bootcmd=bootz 0x60100000 0x64000000 0x60000000
=>

```

Figure 3.5: Comparison between setenv bootargs and printenv bootargs.

- Kernel panic result when there is no rootfs (VFS error message)

```

Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0)
CPU: 0 PID: 1 Comm: swapper/0 Not tainted 5.15.0 #1
Hardware name: ARM-Versatile Express
[<801103a0>] (unwind_backtrace) from [<8010c2c4>] (show_stack+0x10/0x14)
[<8010c2c4>] (show_stack) from [<80867f00>] (dump_stack_lvl+0x40/0x4c)
[<80867f00>] (dump_stack_lvl) from [<80863f64>] (panic+0xfc/0x2e4)
[<80863f64>] (panic) from [<80b01714>] (mount_block_root+0x1bc/0x264)
[<80b01714>] (mount_block_root) from [<80b01b98>] (prepare_namespace+0x148/0x184)
[<80b01b98>] (prepare_namespace) from [<8086c6b8>] (kernel_init+0x14/0x13c)
[<8086c6b8>] (kernel_init) from [<80100130>] (ret_from_fork+0x14/0x24)
Exception stack(0x810a7fb0 to 0x810a7ff8)
7fa0: 00000000 00000000 00000000 00000000
7fc0: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
7fe0: 00000000 00000000 00000000 00000000 00000013 00000000
---[ end Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0) ]-
--

```

Figure 3.6: Kernel panic.

Part C — Root Filesystem & initramfs

- Output of the `ls initramfs/` command

```

phuc@phuc-Inspiron-14-5430:~/embedded_lab1/rootfs/initramfs$ ls -lh ~/embedded_lab1/output
/initramfs.cpio.gz
-rw-rw-r-- 1 phuc phuc 1.1M Sep 12 13:23 /home/phuc/embedded_lab1/output/initramfs.cpio.gz

```

Figure 3.7: Result of running the `ls initramfs/` command.

- Successful QEMU boot log (up to reaching the BusyBox shell)

```
phuc@phuc-Inspiron-14-5430:~/embedded_lab1/kernel/linux-5.15$ qemu-system-arm \
-M vexpress-a9 \
-cpu cortex-a9 \
-m 512M \
-smp 2 \
-nographic \
-kernel ~/embedded_lab1/output/zImage \
-dtb ~/embedded_lab1/output/vexpress-v2p-ca9.dtb \
-initrd ~/embedded_lab1/output/initramfs.cpio.gz \
-append 'console=ttyAMA0,115200 rdinit=/sbin/init mem=512M'
Booting Linux on physical CPU 0x0
Linux version 5.15.0 (phuc@phuc-Inspiron-14-5430) (arm-linux-gnueabi-hf-gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0, GNU ld (GNU Binutils for Ubuntu) 2.46) #1 SMP Sat Sep 12 12:18:55 +07 2026
CPU: ARMv7 Processor [410fc090] revision 0 (ARMv7), cr=10c5387d
CPU: PIPT / VIPT nonaliasing data cache, VIPT nonaliasing instruction cache
OF: fdt: Machine model: V2P-CA9
Memory policy: Data cache writealloc
Reserved memory: created DMA memory pool at 0x4c000000, size 8 MiB
OF: reserved mem: initialized node vram@4c000000, compatible id shared-dma-pool
cma: Reserved 16 MiB at 0x7f000000
Zone ranges:
  Normal [mem 0x0000000060000000-0x000000007fffffff]
Movable zone start for each node
Early memory node ranges
  node 0: [mem 0x0000000060000000-0x000000007fffffff]
Initmem setup node 0 [mem 0x0000000060000000-0x000000007fffffff]
percpu: Embedded 15 pages/cpu s30156 r8192 d23092 u61440
Built 1 zonelists, mobility grouping on. Total pages: 130048
Kernel command line: console=ttyAMA0,115200 rdinit=/sbin/init mem=512M
printk: log_buf_len individual max cpu contribution: 4096 bytes
printk: log_buf_len total cpu_extra contributions: 12288 bytes
printk: log_buf_len min size: 16384 bytes
printk: log_buf_len: 32768 bytes
printk: early log buf free: 14984(91%)
Dentry cache hash table entries: 65536 (order: 6, 262144 bytes, linear)
Inode-cache hash table entries: 32768 (order: 5, 131072 bytes, linear)
mem auto-init: stack:all(zero), heap alloc:off, heap free:off
Memory: 489824K/524288K available (8192K kernel code, 593K rwdata, 1784K rodata, 1024K init, 151K bss, 18080K reserved, 16384K cma-reserved)
SLUB: HWalign=64, Order=0-3, MinObjects=0, CPUs=4, Nodes=1
trace event string verifier disabled
rcu: Hierarchical RCU implementation.
rcu: RCU event tracing is enabled.
rcu: RCU restricting CPUs from NR_CPUS=8 to nr_cpu_ids=4.
```

Figure 3.8: Boot into the BusyBox shell.

- Results of the `uname -a`, `mount`, and `ps` commands in the BusyBox shell

```
/ # uname -a
Linux embedded-lab 5.15.0 #1 SMP Sat Sep 12 12:18:55 +07 2026 armv7l GNU/Linux
/ #
```

Figure 3.9: Running the `uname -a` command.

```
/ # mount
rootfs on / type rootfs (rw,size=244912k,nr_inodes=61228)
none on /proc type proc (rw,relatime)
none on /sys type sysfs (rw,relatime)
none on /tmp type tmpfs (rw,relatime)
/ #
```

Figure 3.10: Result of the `mount` command.

```

/ # ps
PID    USER      TIME  COMMAND
  1  root        0:02   init
  2  root        0:00   [kthreadd]
  3  root        0:00   [rcu_gp]
  4  root        0:00   [rcu_par_gp]
  5  root        0:00   [kworker/0:0-eve]
  6  root        0:00   [kworker/0:0H-ev]
  8  root        0:00   [mm_percpu_wq]
  9  root        0:00   [ksoftirqd/0]
 10  root        0:00   [rcu_sched]
 11  root        0:00   [migration/0]
 12  root        0:00   [cpuhp/0]
 13  root        0:00   [cpuhp/1]
 14  root        0:00   [migration/1]
 15  root        0:00   [ksoftirqd/1]
 17  root        0:00   [kworker/1:0H]
 18  root        0:00   [cpuhp/2]
 19  root        0:00   [migration/2]
 20  root        0:00   [ksoftirqd/2]
 21  root        0:00   [kworker/2:0]
 22  root        0:00   [kworker/2:0H]
 23  root        0:00   [cpuhp/3]
 25  root        0:00   [migration/3]
 26  root        0:00   [ksoftirqd/3]
 27  root        0:00   [kworker/3:0]
 28  root        0:00   [kworker/3:0H]
 29  root        0:00   [kdevtmpfs]
 30  root        0:00   [inet_frag_wq]
 31  root        0:00   [kworker/1:1-eve]
 32  root        0:00   [khungtaskd]
 33  root        0:00   [oom_reaper]
 34  root        0:00   [writeback]
 35  root        0:00   [kcompactd0]
 36  root        0:00   [kblockd]
 37  root        0:00   [ata_sff]
 38  root        0:00   [kworker/u8:1-ev]
 39  root        0:00   [rpciod]
 40  root        0:00   [kworker/u9:0]
 41  root        0:00   [xprtiod]
 42  root        0:00   [kworker/u8:2]
 43  root        0:00   [kworker/0:1H]
 44  root        0:00   [kswapd0]
 45  root        0:00   [nfsiod]
 51  root        0:00   [irq/33-mmci-pl1]
 52  root        0:00   [kworker/0:3-eve]
 53  root        0:00   [kworker/1:2-eve]
 54  root        0:00   [card0-crtc0]
 60  root        0:00   -/bin/sh

```

Figure 3.11: The ps command showing the currently running processes.

Section 4. Analysis and Answers

Provide detailed answers to the 5 questions in Section VI of the assignment. Each answer must give a specific technical explanation, written in your own words (no copying from the internet).

Question 1. Compare arm-linux-gnueabi and arm-linux-gnueabihf

The core difference lies in the ABI (Application Binary Interface), which determines how floating-point numbers (FPU) are handled. `arm-linux-gnueabi` uses soft-float (passing floating-point parameters through general integer registers such as r0-r3 or the stack), while `arm-linux-gnueabihf` uses hard-float (passing parameters directly through dedicated FPU registers such as d0-d7). Cortex-A9 needs `gnueabihf` because this

architecture has built-in FPU hardware (VFPv3/NEON). Using hard-float makes full use of the hardware's capability to speed up computation. If compiled with `gnueabi` for Cortex-A9, the system will still run, but floating-point operations will have to be emulated in software (costing many extra machine cycles) or require additional unnecessary data-transfer instructions, noticeably reducing system performance.

Question 2. Memory map of vexpress-a9 (0x60000000)

[Answer: reason it starts at 0x60000000; effect on bootcmd; consequences of setting the wrong load address]

Reason it starts at `0x60000000`: The memory map of the Versatile Express Cortex-A9 board (emulated by QEMU) strictly specifies, via hardware/device tree, that the first physical RAM space (DDR) starts at address `0x60000000`. Lower addresses are reserved for I/O devices, NOR flash, or internal SRAM. Effect on `bootcmd`: Since RAM is located at `0x60000000`, the `bootcmd` command in U-Boot must load the kernel (zImage), Device Tree (dtb), and `initramfs` into addresses within this range (for example, the kernel is loaded at `0x60000800`). Consequences of setting it wrong: If the kernel is loaded at a lower address (for example `0x0` or `0x40000000`), U-Boot will overwrite data in a Read-Only memory region (such as Flash) or another device's I/O region. This causes a hardware exception (Data Abort Exception) and the system will immediately hang/crash, unable to boot into Linux.

Question 3. Decode MIDR 0x410fc090

Based on the structure of the 32-bit MIDR (Main ID Register) of the ARM architecture, the value `0x410fc090` is decoded into the following fields:

- Bits Implementer: `0x41` (the ASCII code for the character 'A', representing ARM).
- Bits Variant: `0x0` (the main variant version number).
- Bits Architecture: `0xF` (meaning the architecture is identified through the CPUID register, specifically ARMv7 here).
- Bits [15:4] Part number: `0xC09` (the processor core identifier code). The code `0xC09` corresponds to the ARM Cortex-A9 core CPU.
- Bits [3:0] Revision: `0x0` (the minor patch/revision number).

Question 4. Static linking BusyBox

Why static linking is needed: Static linking embeds all the code of the dependent libraries (such as the C library) directly into the BusyBox executable. The benefit is an extremely compact rootfs that works independently right away without needing a `/Lib` directory structure containing shared libraries. If dynamic linking is used, which `.so` files are needed, and where they are located: if dynamic linking is chosen, BusyBox will need at least 3 basic shared-library files to run:

1. `libc.so.6` (Standard C library)
2. `libm.so.6` (Math library)
3. `ld-linux-armhf.so.3` (Dynamic linker/loader for loading other `.so` files) These files are located inside the cross-compiler toolchain on the host machine, usually at the path: `[toolchain_install_dir]/arm-linux-gnueabihf/libc/lib/`. When building the rootfs, you must manually copy these files into the `/Lib` directory on the target.

Question 5. SMP in the kernel config (-smp 2)

What else the kernel needs to initialize: To run multiple cores (Symmetric Multiprocessing), the kernel needs to initialize and configure the GIC (Generic Interrupt Controller) to route interrupts to each core, and configure the

SCU (Snoop Control Unit) to maintain cache coherency between cores. Most importantly, CPU0 must send an IPI (Inter-Processor Interrupt) signal to "wake up" CPU1 from its startup wait state. Boot log comparison (2 representative differing lines):

- When using `-smp 2`, the dmesg log will show:
 1. `CPU1: thread -1, cpu 1, socket 0, mpidr 80000001` (Initializes and configures the parameters for the secondary core).
 2. `smp: Brought up 1 node, 2 CPUs` (Confirms the system has brought up both CPUs).
- When using `-smp 1`, the log will stop at:
 1. There will be no log line mentioning booting **CPU1**.
 2. It will record: `smp: Brought up 1 node, 1 CPUs`.

Section 5. Observations & Difficulties Encountered

Difficulty 1 Description of the error encountered: When I ran the command to build the kernel image, the terminal displayed the error message `make[2]: *** No rule to make target 'zImage'.` **Cause:** This error occurred because I had not configured the build system to recognize the target hardware architecture as ARM. **Fix:** I checked the environment variables again and ran the `export ARCH=arm` command before rebuilding.

Difficulty 2 Description of the error encountered: When I booted QEMU with the newly built kernel, the system froze and reported the error `Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(1,0)`. **Cause:** I realized this error was because the system did not have an `initramfs` to mount the root filesystem, or the parameter passed in was incorrect. **Fix:** I proceeded to create the Root Filesystem using `BusyBox`, packaging it into the file `initramfs.cpio.gz` following the instructions. When rebooting, I added the parameter `-initrd` pointing correctly to this compressed file, and carefully rechecked the `root=` variable in `bootargs`.

General observations The Linux operating system and working entirely in a command-line environment were still quite new to me, which led to some difficulties at first, most notably the error when building the file `zImage`. However, after completing the lab, I gained a clearer understanding of the steps involved in setting up a complete boot sequence for an embedded Linux system, from running the bootloader and loading the kernel to reaching userspace. Typing the commands myself, reading the error messages, and figuring out how to debug them helped me build carefulness and gain a more practical view of software systems.